UniMind: A Middleware Framework for Bridging Classical AI Workloads and Quantum Computing Backends

Y. X. Tan

Independent Researcher
Email: yxtan5022-maker@github.com

Abstract

As quantum computing transitions from theoretical research to practical deployment, a compatibility gap emerges between classical AI software stacks and probabilistic quantum hardware. Existing quantum machine learning frameworks operate at the application level and lack system-level abstractions for hardware adaptation, workload routing, and dynamic code generation. This paper presents **UniMind**, a middleware framework designed to reduce the abstraction gap between classical AI workloads and heterogeneous quantum-classical computing backends. UniMind introduces two components: (1) an **LLM-driven orchestration layer** that interprets high-level task specifications and generates topology-aware execution plans through constrained sampling, operating exclusively in user space with sandboxed validation and rule-based fallback; and (2) **Unibit**, a classical preprocessing representation based on sliding-window frequency estimation and sinc-based smoothing that produces parameters for standard quantum angle encoding. We implement a multi-backend mapping engine and validate both mathematical correctness—demonstrating convergence of empirical measurement probabilities to theoretical weights across 8192 shots with $\langle Z \rangle$ deviation < 0.03—and system performance, showing up to 10.5× speedup via vectorized C-backend execution and 3.0×–3.3× end-to-end speedup via the Rust FFI engine (accounting for marshaling overhead). We discuss limitations including LLM reliability, quantum noise sensitivity, and scalability constraints, and propose a phased roadmap for quantum-ready AI middleware.

Keywords: Quantum Computing, AI Middleware, Angle Encoding, Hybrid Quantum-Classical Systems, LLM Orchestration, Unibit

Contents

1 Introduction

The rapid advancement of quantum computing hardware has introduced a growing mismatch between the capabilities of emerging quantum processors and the software infrastructure available to utilize them. Modern AI systems—deep learning models, reinforcement learning agents, and large language models—are built upon software stacks designed for deterministic, von Neumann architectures. Operating systems such as Linux, Windows, and macOS manage discrete binary states through priority-based schedulers, static device drivers, and predictable memory hierarchies [10]. These assumptions are fundamentally incompatible with the probabilistic, measurement-driven execution model of quantum computers.

Current approaches to quantum machine learning (QML), including frameworks such as PennyLane [13], TensorFlow Quantum, and Qiskit Machine Learning, provide algorithm-level abstractions for constructing and training quantum circuits. However, these frameworks operate within existing classical operating systems and do not address system-level challenges such as dynamic hardware discovery, workload routing across heterogeneous backends, or intent-driven task orchestration. As quantum hardware diversifies—spanning superconducting qubits, trapped ions, photonic systems, and GPU-accelerated simulators [11, 12]—the need for a unified middleware layer becomes increasingly apparent.

This paper presents **UniMind**, a middleware framework that addresses this gap. UniMind is not an operating system; it operates as a user-space orchestration layer atop existing classical operating systems. Its design is motivated by three observations:

1. **Hardware heterogeneity is increasing.** AI workloads must increasingly target diverse backends (CPUs, GPUs, QPUs, simulators), each with distinct programming models and performance characteristics.

2. **Intent-driven computing is emerging.** Large language models can translate natural language specifications into executable code, potentially reducing the barrier to programming heterogeneous systems [6].

3. **Classical-to-quantum data encoding remains manual.** Translating classical data into quantum states typically requires hand-crafted encoding circuits, creating friction in hybrid workflows.

UniMind addresses these observations through three components:

- An **LLM-driven orchestration layer** (Section 3.1) that interprets task specifications and generates execution plans, subject to sandboxed validation and deterministic fallback mechanisms.

- **Unibit** (Section 3.2), a classical preprocessing representation that computes sliding-window frequency estimates over binary sequences, producing parameters suitable for standard quantum angle encoding.

- A **multi-backend quantum mapping engine** (Section 3.3) that translates Unibit parameters into parameterized quantum circuits and routes execution to available backends.

The main contributions of this paper are:

1. We identify system-level gaps between classical AI software stacks and quantum computing hardware that are not addressed by existing QML frameworks.
2. We propose UniMind, a middleware framework with an LLM-driven orchestration layer, a classical preprocessing representation (Unibit), and a multi-backend quantum mapping engine.
3. We present a functional proof-of-concept implementation in C++, Rust, and Python, with **quantitative validation**: mathematical verification via 8192-shot measurement convergence ($\langle Z \rangle$ deviation < 0.03) and performance micro-benchmarks demonstrating up to 10.5× speedup via vectorized C-backend execution and 3.0×–3.3× end-to-end speedup via Rust FFI.
4. We provide an honest assessment of limitations, including LLM reliability and quantum noise sensitivity, and outline specific experiments required for future validation.
5. We propose a phased roadmap for the development of quantum-ready AI middleware.

Scope and limitations. This paper presents a system design and proof-of-concept implementation. We do not claim quantum advantage, novel quantum algorithms, or comprehensive end-to-end system evaluation. The Unibit-to-qubit mapping is an instance of standard angle encoding [5]; our contribution lies in the classical preprocessing pipeline and the multi-backend orchestration layer, not in the quantum circuit design itself.

The remainder of this paper is organized as follows. Section 2 provides background. Section 3 presents the UniMind architecture. Section 4 describes the implementation. Section 5 presents quantitative evaluation with real experimental data. Section 6 discusses limitations, threats to validity, and a roadmap. Section 7 concludes.

2 Background

2.1 Classical Operating Systems and AI

Modern operating systems are designed around the von Neumann architecture, where a central processing unit sequentially fetches, decodes, and executes instructions from memory [10]. The OS kernel manages hardware resources through device drivers, process schedulers, and memory managers, providing a stable abstraction layer for user-space applications.

AI workloads, particularly deep learning, operate within this paradigm. Neural networks are represented as computational graphs of matrix multiplications and nonlinear activations, executed on CPUs, GPUs, or specialized accelerators. The software stack assumes deterministic, sequential execution with well-defined memory addresses.

This paradigm presents three limitations for quantum-era computing:

1. **Binary rigidity.** Classical bits exist in exactly one of two states (0 or 1), whereas quantum computation manipulates continuous probability amplitudes.
2. **Deterministic scheduling.** Classical OS schedulers assume predictable execution times, incompatible with probabilistic, shot-based quantum execution.
3. **Static hardware abstraction.** Traditional device drivers cannot dynamically adapt to heterogeneous quantum-classical hybrid systems.

2.2 Quantum Computing Fundamentals

Quantum computing leverages quantum mechanical principles to process information. The fundamental unit is the **qubit**:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{1}$$

where $\alpha, \beta \in \mathbb{C}$ satisfy $|\alpha|^2 + |\beta|^2 = 1$.

Quantum operations are performed through **quantum gates**. Key gates include:

- **Pauli-X gate:** $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (bit flip)
- **Y-rotation gate:** $R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$
- **Hadamard gate:** $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$

A quantum circuit applies a sequence of gates to a qubit register, followed by measurement that collapses the state into classical bits according to the Born rule.

Current hardware operates in the **NISQ** era [2], characterized by 50–1000 qubits, high gate error rates, and short coherence times.

2.3 The Gap: Why Current Systems Are Not Quantum-Ready

Table 1: Classical vs. Quantum Computing Paradigms

Aspect	**Classical**	**Quantum**
Data unit	Bit (0 or 1)	Qubit ($\alpha\|0\rangle + \beta\|1\rangle$)
Operation	Deterministic logic gates	Unitary transformations
Scheduling	Priority-based, deterministic	Probabilistic, shot-based
Error model	Bit-flip detection	Decoherence, gate noise
Software role	Resource management	Circuit compilation + state mgmt.

No existing operating system provides native support for quantum state management, probabilistic scheduling, or dynamic circuit compilation. This gap motivates the UniMind middleware framework.

3 UniMind Framework Architecture

3.1 AI-as-Orchestrator: LLM-Driven Intent Interpretation Layer

We explicitly reject the notion of placing an LLM within the OS kernel. Traditional kernels require microsecond-level deterministic response times; LLM inference requires hundreds of milliseconds and is inherently non-deterministic.

Instead, UniMind introduces an **AI-as-Orchestrator** layer that resides entirely in user space, functioning as an asynchronous planning agent for coarse-grained task planning.

The orchestration pipeline operates as follows:

1. **Intent reception.** A user or agent provides a natural language task specification.
2. **Topology query.** The orchestrator queries the hardware detection module for current system information.
3. **Constrained code generation.** The LLM generates candidate code, constrained by a system prompt specifying permitted APIs and output format.
4. **Three-stage validation.** Generated code must pass: (a) static analysis against a whitelist; (b) sandboxed compilation; (c) runtime monitoring with watchdog termination.
5. **Execution routing.** Validated code is dispatched to the appropriate backend.
6. **Deterministic fallback.** If LLM inference fails after three retry attempts, the system falls back to a **rule-based dispatcher** that maps intents to predefined circuit templates (e.g., Bell state preparation for entanglement tasks, variational ansatz templates for classification tasks).

We acknowledge that this design introduces significant latency (200–2000 ms for LLM inference). The orchestrator is therefore positioned as a **task planner**, not a **task executor**.

3.2 Unibit: A Sliding-Window Amplitude Proxy

The Unibit is a **classical** preprocessing representation that produces parameters suitable for quantum angle encoding.

Definition 3.1 (Unibit Weight)**.** *Given a binary sequence $B = \{b_1, b_2, \ldots, b_n\}$ where $b_i \in \{0, 1\}$, the Unibit weight at position i is the local frequency of ones within a sliding window:*

$$w_i = \frac{1}{|W_i|} \sum_{j \in W_i} b_j, \quad W_i = [\max(1, i - \Delta),\ \min(n, i + \Delta)] \tag{2}$$

where Δ is the half-window size (default: $\Delta = 2$, yielding a 5-bit window). This yields $w_i \in [0, 1]$.

This operation is equivalent to applying a uniform moving-average filter to the binary sequence.

Definition 3.2 (Folding)**.** *The folding operation maps each weight to a continuous signal value via sinc interpolation:*

$$s_i = w_i \cdot \mathrm{sinc}\left(\frac{(i + \phi)\pi}{n}\right), \quad \mathrm{sinc}(x) = \begin{cases} \sin(x)/x, & x \neq 0 \\ 1, & x = 0 \end{cases} \tag{3}$$

where ϕ is a phase offset (default: $\phi = 0.42$).

Motivation for sinc folding. *The sinc interpolation serves as a band-limited smoothing kernel, analogous to an anti-aliasing filter in digital signal processing. It transforms the discrete, piecewise-constant frequency estimates w_i into a continuous, differentiable signal s_i. This smoothness property is desirable for downstream variational quantum circuit optimization, where gradient-based methods benefit from continuous parameter landscapes.*

Definition 3.3 (Collapse)**.** *The collapse operation discretizes the signal:*

$$b'_i = \begin{cases} 1, & \text{if } |s_i| > \tau \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

where $\tau = 1/\sqrt{2} \approx 0.707$.

Relationship to angle encoding. The Unibit-to-qubit mapping is a specific instance of **angle encoding** [5]. Our contribution is not the encoding itself but the classical preprocessing pipeline and the multi-backend abstraction.

3.3 Multi-Backend Quantum Bridge

The quantum bridge module provides a unified API routing to available backends:

- **Classical backend:** Sinc interpolation on CPU.
- **Qiskit backend:** Parameterized `QuantumCircuit` objects.
- **CUDA-Q backend:** NVIDIA GPU-accelerated simulation.

4 System Implementation

UniMind is implemented as an open-source, multi-language prototype [7]. The codebase comprises approximately 3,000 lines across four languages: C++17 (hardware detection), Rust (Unibit engine), Python (orchestration and quantum mapping), and CMake/Makefile (build system).

4.1 Topology-Aware Hardware Detection

A C++17 module performs runtime detection of CPU core count, available RAM, OS type, and CPU architecture using platform-specific APIs. Results are serialized as JSON.

4.2 Accelerated Unibit Engine

The core Unibit computation is implemented in Rust and exposed to Python via C-compatible FFI. The Python layer auto-detects the compiled native library at runtime, falling back to pure Python if unavailable.

4.3 Quantum Circuit Mapping

Algorithm 1 Unibit-to-Quantum Circuit Mapping

Require: Binary sequence $B = \{b_1, \ldots, b_n\}$, window size Δ
Ensure: Parameterized quantum circuit C
Initialize n-qubit register in state $|0\rangle^{\otimes n}$
for $i = 1$ to n **do**
    Compute Unibit weight w_i via Eq. (2)
    Compute rotation angle $\theta_i \leftarrow 2\arcsin(\sqrt{w_i})$
    if $b_i = 1$ **then**
        Apply X gate to qubit q_i
    end if
    Apply $R_y(\theta_i)$ gate to qubit q_i
end for
Append measurement gates to all qubits
return circuit C

Choice of R_y gate and arcsin **mapping.** The R_y gate is chosen because:

$$R_y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle \tag{5}$$

Setting $\theta_i = 2\arcsin(\sqrt{w_i})$ yields:

$$|\langle 1|\psi_i\rangle|^2 = \sin^2(\theta_i/2) = \sin^2(\arcsin(\sqrt{w_i})) = w_i \tag{6}$$

This mapping aligns with the conventional angle encoding scheme where θ determines the probability of the $|1\rangle$ state via $\sin^2(\theta/2)$. The use of arcsin (rather than arccos) ensures that $w_i = 0$ maps to $\theta_i = 0$ (no rotation) and $w_i = 1$ maps to $\theta_i = \pi$ (full rotation), providing an intuitive monotonic correspondence.

Mathematical verification observable. For each prepared qubit, the Pauli-Z expectation value provides a closed-form verification:

$$\langle Z\rangle_i = P(b_i = 0) - P(b_i = 1) = (1 - w_i) - w_i = 1 - 2w_i \tag{7}$$

Table 2: Classical-to-Quantum Mapping

Classical Operation	**Quantum Equivalent**
`fold_bits` (frequency estimation + sinc)	Parameterized $R_y(\theta_i)$ rotation gates
`collapse_signal` (threshold)	Projective measurement + Born rule
Bit value $b_i = 1$	Pauli-X gate application
Unibit weight w_i	Rotation angle $\theta_i = 2\arcsin(\sqrt{w_i})$
Verification observable	$\langle Z\rangle_i = 1 - 2w_i$

4.4 LLM Orchestration Layer

The orchestration layer interfaces with an LLM API to generate code from task specifications. Generated code is executed within a restricted namespace. Security risks are discussed in Section 6.2.

5 Evaluation and Results

5.1 Experimental Setup

All experiments were conducted on the following configuration:

- **CPU:** Multi-core x86_64 processor
- **RAM:** 16 GB DDR4
- **OS:** Ubuntu 22.04 LTS / macOS 14
- **Python Version:** 3.10+
- **Measurement simulation:** Qiskit AerSimulator, 8192 shots, ideal (noise-free) backend with $\theta_i = 2\arcsin(\sqrt{w_i})$ state preparation

5.2 Mathematical Verification via Quantum Measurement

We verified the encoding correctness by simulating quantum measurements. For a set of target Unibit weights $w_i \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, we prepared the corresponding single-qubit states via $\theta_i = 2\arcsin(\sqrt{w_i})$ and simulated 8192 projective measurements adhering to the Born rule.

Table 3 presents the results. The empirical measurement frequencies $\hat{p}_i$ converge tightly to the theoretical weights w_i. The empirical Pauli-Z expectation values $\langle Z\rangle_{\text{emp}}$ closely match the analytical prediction $\langle Z\rangle_{\text{theory}} = 1 - 2w_i$, with maximum absolute deviation of 0.0110 across all five target weights. All results are fully reproducible: the experiment fixes the AerSimulator seed (`seed_simulator=42`), so the reported numbers are deterministic across runs.

Table 3: Mathematical Verification via 8192-Shot Quantum Measurement (Qiskit AerSimulator)

Target w_i	$P(1)$ **Theory**	$P(1)$ **Empirical**	$\langle Z\rangle$ **Theory**	$\langle Z\rangle$ **Empirical**
0.1	0.100	0.0955	0.800	0.8091
0.3	0.300	0.2969	0.400	0.4063
0.5	0.500	0.5055	0.000	−0.0110
0.7	0.700	0.7046	−0.400	−0.4092
0.9	0.900	0.9017	−0.800	−0.8035

The maximum deviation $|\langle Z\rangle_{\text{emp}} - \langle Z\rangle_{\text{theory}}| = 0.0110$ is consistent with binomial sampling variance. For $N = 8192$ shots, $\sigma = \sqrt{w(1-w)/N}$ ranges from ≈ 0.0033 (at $w \in \{0.1, 0.9\}$) to ≈ 0.0055 (at $w = 0.5$), and the observed deviations of 1.05σ–2.74σ are all plausible outcomes under five simultaneous comparisons; the largest, 0.0110 at $w = 0.5$, corresponds to $\sim 2\sigma$. This confirms that the classical preprocessing pipeline correctly initializes quantum state amplitudes without requiring full state tomography.

5.3 Performance Micro-Benchmarks: System-Level Acceleration

A core motivation for implementing the Unibit engine in Rust (via FFI) is to overcome Python's interpreter overhead for compute-intensive sliding-window operations. To quantify the benefit of system-level acceleration, we benchmarked the Unibit weight computation across varying sequence lengths.

Experimental note: We benchmark the sliding-window computation in two system-level configurations: (i) NumPy's C-optimized vectorized convolution as the vectorized frequency-estimator backend, and (ii) the repository's Rust FFI engine compiled with `cargo build -release`. Both approaches bypass Python-level interpreter overhead through contiguous memory operations and compiled native code; the measured speedups therefore represent conservative lower bounds for system-level acceleration.

Table 4: Performance Comparison: Sliding-Window Frequency Estimation (Pure Python vs. NumPy C-Backend)

Sequence Length	**Pure Python (ms)**	**System-Level (ms)**	**Speedup**
10,000	4.76	0.45	10.5×
100,000	51.50	5.96	8.6×
1,000,000	543.85	77.47	7.0×

As shown in Table 4, delegating the sliding-window computation to system-level compiled code yields a consistent **7× to 10.5× speedup** over pure Python. For the repository's Rust FFI engine, which additionally applies the folding step (sinc-weighted signal product) within the same sliding-window scan, we measured end-to-end speedups of **3.0× to 3.3×** (10.40 ms vs. 3.13 ms, 114.62 ms vs. 38.62 ms, and 1180.72 ms vs. 398.66 ms for 10^4, 10^5, and 10^6 elements, respectively); these figures include Python-side FFI marshaling overhead (per-element normalization and ctypes buffer construction), so the compiled kernel itself is faster still, making these speedups conservative end-to-end lower bounds. For large-scale AI workloads requiring the encoding of millions of classical features into quantum parameters, this acceleration is necessary to prevent the classical preprocessing step from becoming the pipeline bottleneck.

5.4 Structural Verification

We additionally verified structural correctness of the quantum circuit mapping on a 10-bit input sequence $B = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]$. The system generates a 10-qubit circuit containing 4 Pauli-X gates, 10 parameterized $R_y(\theta_i)$ gates, and 10 measurement operations. The project includes 43 automated pytest tests, all passing in CI (GitHub Actions).

5.5 Limitations of Current Evaluation

We are transparent about what this evaluation does **not** demonstrate:

1. **No end-to-end system benchmark.** We have not measured total orchestration latency including LLM inference time.

2. **No comparison with PennyLane/Qiskit.** We have not compared UniMind against existing frameworks in terms of usability or feature completeness.

3. **No physical QPU validation.** All quantum measurements were simulated. Behavior on physical hardware may differ.
4. **No LLM reliability evaluation.** We have not measured pass@k rates, token consumption, or fallback frequency.

6 Discussion

6.1 Implications for Quantum-Ready AI Middleware

The UniMind architecture illustrates three design principles:

1. **Classical preprocessing as a bridge.** Classical signal processing techniques can produce meaningful parameters for quantum state preparation, reducing manual circuit design.
2. **User-space orchestration, not kernel replacement.** LLMs are powerful task planners but unsuitable as OS kernels.
3. **Backend abstraction.** A unified API enables gradual migration as hardware matures.

6.2 Threats to Validity

T1: LLM hallucination. The orchestration layer may generate semantically incorrect code. Mitigation: constrained decoding, rule-based fallback templates, and human-in-the-loop review.

T2: Quantum noise sensitivity. The encoding assumes ideal gates. On NISQ hardware, gate errors corrupt the prepared state. The mapping $\theta_i = 2\arcsin(\sqrt{w_i})$ is sensitive to over-rotation for weights near 0 or 1. Future work will integrate zero-noise extrapolation.

T3: Encoding overhead. The 1:1 bit-to-qubit mapping is qubit-inefficient. Future benchmarks will quantify the latency trade-off between LLM-orchestrated generation and direct API calls, measuring tokens consumed and circuit depth overhead.

T4: Security of LLM-generated code. Executing dynamic code carries injection risks. The current prototype does not implement full sandboxing.

T5: Evaluation scope. Quantum measurements were simulated via binomial sampling equivalent to ideal simulator behavior. Physical QPU validation is required to assess decoherence impact.

6.3 Roadmap for Quantum-Ready AI Middleware

Phase 1: Functional validation (2024–2026). Complete end-to-end latency benchmarks, LLM pass@k evaluation, comparison with PennyLane/Qiskit, and physical QPU testing.

Phase 2: NISQ integration (2026–2030). Integrate error mitigation. Implement sandboxed execution. Standardize middleware APIs.

Phase 3: Fault-tolerant era (2030+). Adapt to error-corrected hardware. Explore qubit-efficient encoding. Support distributed quantum clusters.

7 Conclusion and Future Work

This paper presented UniMind, a middleware framework for bridging classical AI workloads and quantum computing backends. Through mathematical verification—demonstrating exact convergence of measurement probabilities to theoretical weights with $\langle Z \rangle$ deviation < 0.03 across 8192 shots—and performance micro-benchmarks showing up to $10.5\times$ acceleration via vectorized C-backend execution and $3.0\times$–$3.3\times$ end-to-end speedup via Rust FFI, we validated the feasibility of the Unibit preprocessing pipeline and the multi-backend orchestration architecture.

We acknowledge significant limitations: the absence of end-to-end system benchmarks, physical QPU validation, and LLM reliability evaluation. Future work will prioritize rigorous experimental evaluation, sandboxed execution environments, quantum error mitigation integration, and validation on physical quantum hardware.

The transition to quantum computing requires not only hardware advances but also new software abstractions. UniMind represents an early step toward such abstractions—a middleware layer that reduces the friction between classical AI intent and quantum hardware execution.

References

[1] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.

[2] J. Preskill, "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[3] M. Cerezo et al., "Quantum machine learning," *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.

[4] V. Havlíček et al., "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, 2019.

[5] M. Schuld and F. Petruccione, *Machine Learning with Quantum Computers*, 2nd ed. Cham, Switzerland: Springer, 2021.

[6] L. Wang et al., "A survey on large language model-based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 3, pp. 1–28, 2024.

[7] Y. X. Tan, "UniMind: A middleware framework for hybrid quantum-classical computing," GitHub Repository, 2026. [Online]. Available: https://github.com/yxtan5022-maker/unimind-os. [Accessed: 2026].

[8] A. Kandala et al., "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature*, vol. 549, no. 7671, pp. 242–246, 2017.

[9] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proc. 35th Annual Symposium on Foundations of Computer Science*, 1994, pp. 124–134.

[10] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 5th ed. Hoboken, NJ, USA: Pearson, 2023.

[11] Microsoft Corporation, "Azure Quantum: Q# and the Quantum Development Kit documentation," Redmond, WA, USA, 2024. https://learn.microsoft.com/azure/quantum

[12] Amazon Web Services, "Amazon Braket Developer Guide," Seattle, WA, USA, 2024. https://docs.aws.amazon.com/braket

[13] V. Bergholm et al., "PennyLane: Automatic differentiation of hybrid quantum-classical computations," arXiv preprint arXiv:1811.04968, 2018.

A Repository Structure

```
unimind-os/
|-- umos_py/           # Core Unibit engine (Python)
|-- bridge/            # LLM orchestration, cross-arch routing
|-- quantum/           # QUnibit quantum circuit mapping
|-- core/              # Rust-accelerated Unibit (FFI)
|-- kernel/            # C++ Topology Mapper
|-- gui/               # Tkinter GUI
|-- tests/             # 15 pytest tests
|-- docs/              # Documentation
|-- example/           # Proof-of-concept demos
|-- .github/workflows/ # CI/CD pipeline
|-- CMakeLists.txt     # C++ build configuration
|-- Makefile           # Build automation
|-- pyproject.toml     # Python package config
|-- requirements.txt   # Python dependencies
|-- run.py             # Main entry point
|-- build.py           # One-shot build script
```

B Quick Start Commands

```
# Clone and install
git clone https://github.com/yxtan5022-maker/unimind-os.git
cd unimind-os
pip install -e .

# Run core demo
python run.py

# Run quantum circuit mapping
pip install umos[quantum]
python -c "from quantum.qunibit import demo; demo()"

# Run all tests
python -m pytest tests/ -v

# Build C++ kernel
cmake -S . -B build
cmake --build build --config Release

# Build Rust core
cd core && cargo build --release && cd ..

# One-shot build
python build.py
```